

XShift: FPGA-efficient Binarized LLM with Joint Quantization and Sparsification

Shuai Zhou, Huinan Tian, Sisi Meng, Jianli Chen, Jun Yu, and Kun Wang[†]

State Key Lab of Integrated Chips & Systems, and School of Microelectronics, Fudan University, Shanghai, China

[†]Corresponding author: kun.wang@ieee.org

Abstract—Binarization is a promising approach to significantly reduce computational complexity by replacing multiplications with hardware-efficient XNOR operations. However, the binarization of LLM activations often leads to severe accuracy degradation, while weight-only binarization fails to eliminate multipliers due to the Self-Attention mechanism. Furthermore, LLMs exhibit distinctive channel-level data distribution characteristics and differing computational and memory requirements between the Pre-fill and Decoding stages, necessitating a specialized inference framework.

In response, we introduce *XShift*, an algorithm-hardware co-design framework optimized for efficient binarized LLM inference on FPGAs. *XShift* incorporates three key contributions: (1) a hardware-friendly XNOR-Shift Encoding (XSE) format that transforms traditional multiplications into XNOR and shift operations, ensuring scalability and precision; (2) Hardware Adaptive Outlier and Sparsity (HAOS) techniques, which exploit channel-level data distribution and systolic array architectures for optimized quantization and sparsification; and (3) a dedicated hardware accelerator featuring an XNOR-Shift Systolic Array (XSSA) and an enhanced Base-2 SoftMax Converter (BSMC), designed to address the specific computational demands of binarized LLMs. Experimental evaluations on the Alveo U280 and U50 FPGA demonstrate that *XShift* achieves a 10-15× reduction in DSP resource usage while surpassing existing accelerators and GPUs in inference performance. Specifically, *XShift* delivers an average speedup of 4.17-4.76× and a 14.29-6.95× improvement in energy efficiency, alongside lower perplexity compared to other low-precision LLM techniques. These results underscore the potential of *XShift* for edge deployment of LLMs.

I. INTRODUCTION

The evolution of Large Language Models (LLMs) since the introduction of GPT-1 [1] in 2018, which contained approximately 0.1 billion parameters, has been extraordinary. Within just six years, SOTA models like LLaMA-3 [2] now encompass up to 405 billion parameters. Utilizing BF16 precision, such massive models require at least 810 GB of GPU memory, typically distributed across two systems equipped with 8 Nvidia H100 GPUs each. This exponential growth in size and computational demands presents substantial challenges for deployment, particularly in resource-constrained environments.

To mitigate these challenges, various model compression techniques have been developed, including quantization [3]–[11] and sparsification [12]–[14]. Additionally, hardware accelerators based on FPGA [15]–[17] and ASIC [18]–[22] have been explored. For example, FlightLLM [15] integrates mixed precision and sparsity into an accelerator, but its additional hardware overhead and limited compatibility with systolic arrays constrain its flexibility. Moreover, its inability to quantize activations below 8 bits remains a bottleneck.

Binarization, a more hardware-friendly alternative to low-precision quantization, has emerged as a promising technique [3], [4], [23], [24]. However, current efforts in binarizing Transformers have been limited to models with relatively few parameters, such as BERT

[23] and ViT [24]. While weight binarization has been partially achieved for larger LLMs [3], [4], it fails to eliminate attention matrix multiplications during inference. Furthermore, high-precision activations, particularly in the decoding stage where KV cache is prominent, substantially increase hardware storage demands. Fully binarizing both weights and activations often results in severe accuracy degradation, and the lack of specialized hardware support has hindered its practical application.

In addition, LLMs exhibit distinct computational and memory characteristics between the Pre-fill and Decoding stages [25], [26]. In edge computing scenarios, where resources are limited, achieving efficiency is critical. Users expect minimal latency during the Pre-fill stage and token output rates during the Decoding stage that align with natural reading speeds of 150–300 words per minute [27]. Designing inference frameworks that cater to these requirements while optimizing hardware utilization remains a significant challenge.

In this paper, we propose **XShift**, an algorithm-hardware co-design framework tailored for efficient binarized LLM inference on FPGAs. The key contributions of our work are summarized as follows:

- We introduce a hardware-efficient XNOR-Shift Encoding (XSE) format that eliminates the need for DSPs in matrix computations. The XSE format supports both standard low-bit quantization and an extended version for higher representational range.
- We propose Hardware Adaptive Outlier and Sparsity (HAOS) techniques that leverage LLM-specific channel-level data distributions and systolic array architectures to achieve highly compressed yet accurate models.
- We develop a dedicated hardware accelerator featuring an XNOR-Shift Systolic Array (XSSA) with three computation modes, optimized for LLM inference. Additionally, we design a Base-2 SoftMax Converter (BSMC) for efficient SoftMax processing on FPGA.
- Through extensive experimentation, *XShift* achieves higher accuracy compared to other low-precision LLM approaches. It also outperforms leading accelerators and GPUs in terms of inference speed and energy efficiency.

II. BACKGROUND AND MOTIVATION

A. Large Language Models

Traditional metrics such as end-to-end (E2E) latency and throughput are insufficient for evaluating the inference performance of Large Language Models (LLMs) due to the distinct characteristics of the Pre-fill and Decoding stages. Instead, metrics like Time to First Token (TTFT) and Time Between Tokens (TBT) are more appropriate [26]. As illustrated in Fig. 1(a), TTFT measures the latency from the reception of a user request to the generation of the first output token, where lower values are critical for a responsive user experience. TBT quantifies the interval between the generation of consecutive tokens during the Decoding stage, aligning with typical reading speeds.

In real-world edge deployment scenarios, user requests are sporadic and often unbatchable, and the number of iterations in the

This work was financially supported in part by the National Key Research and Development Program of China under Grant 2021YFA1003602, in part by Project of Shanghai EC under Grant 24KXZNA12, in part by National Natural Science Foundation of China (NSFC) Research Project 92373207. The computations in this research were performed using the CFFF platform of Fudan University.

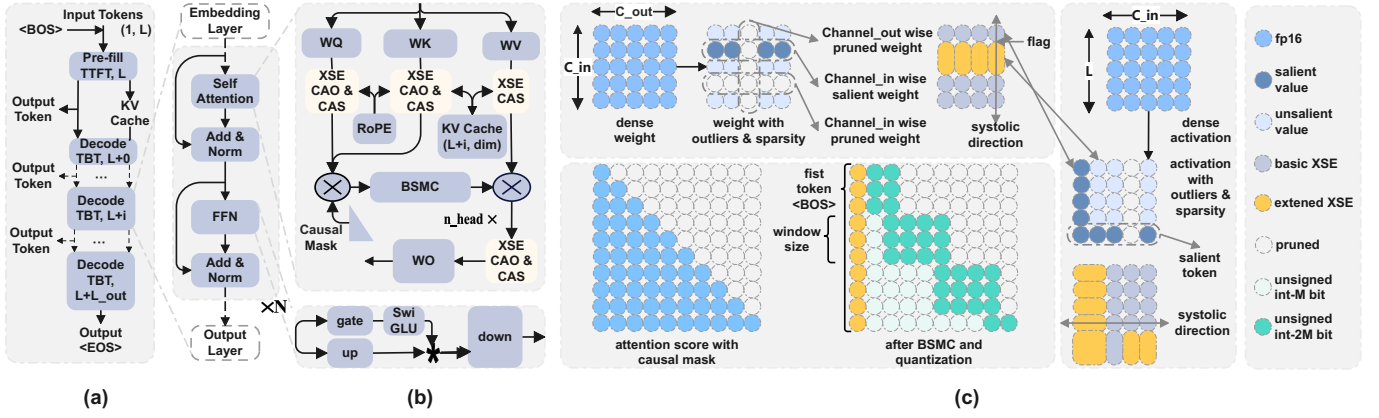


Fig. 1. Network architecture of the proposed *XShift*: (a) LLM inference with one Pre-fill stage and iterative Decoding stage; (b) Detailed attention mechanism of our *XShift* model; (c) Weights and activations optimized using HAOS techniques.

Decoding stage varies significantly across requests. Unlike data center environments, where batching multiple user requests optimizes GPU utilization, edge devices frequently encounter discontinuous requests [25]. Batching under these conditions can introduce unnecessary delays, as new requests need to wait for ongoing Decoding processes to complete. Moreover, even if different inputs are batched together, the variability in Decoding iteration counts across requests can result in hardware underutilization, reducing overall inference efficiency.

As depicted in Fig. 2, an effective edge inference framework must accommodate three operational modes. During the Pre-fill stage, the framework should minimize the average TTFT for each user request by optimizing weight reuse through fine-grained batching. In the Decoding stage, the framework must balance energy efficiency with maintaining a TBT speed suitable for natural reading rates.

B. LLM Compression Techniques

Quantization of both activations and weights is a widely adopted approach to enhance hardware efficiency. However, it poses significant challenges, particularly in addressing outliers that deviate markedly from the typical data distribution. Techniques as [8], [28], [29] manage outliers by isolating them into sparse matrices for specialized processing. Methods like [20], [22] employ paired strategies to reduce the overhead of managing outliers separately, while OPAL [21] uses mixed precision and specialized formats to manage a fixed number of outliers per block. Channel-level observations have also inspired approaches [6], [7], [10], [30], which employ channel-wise scaling to shift quantization difficulty from activations to weights.

Sparsity in LLMs is another promising avenue for compression. [12], [14] utilize unstructured sparsity to minimize performance degradation during compression. Meanwhile, structured pruning approaches [13], [31] focus on improving hardware compatibility. Joint sparsity and quantization techniques, as exemplified by JSQ [32], have shown significant promise in achieving high compression rates with minimal accuracy loss.

Among all compression methods, binarization [3], [4], [23], [24] stands out for its superior compression ratios and its ability to eliminate multiplications, replacing them with more efficient operations. While Binary Neural Network (BNN)-specific accelerators [33], [34] have demonstrated potential, they often lack support for LLM architectures, underscoring the need for tailored hardware designs that fully exploit the advantages of LLM binarization.

C. LLM Deployment on Edge Devices

Complementing LLM algorithmic optimizations, various LLM accelerators based on [15]–[17] and ASIC [18]–[22] have been pro-

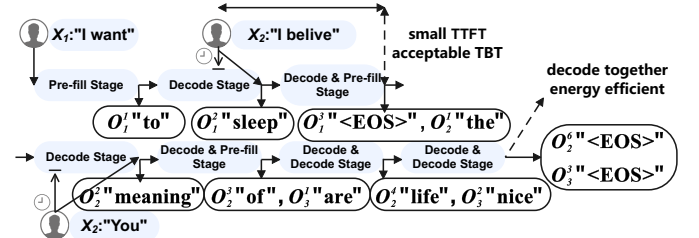


Fig. 2. Illustration of *XShift*'s effectiveness in handling discontinuous requests using the XSSA with three computation modes.

posed to enhance deployment efficiency. Previous works have focused on designing accelerators to speed up inference in Transformer-based neural networks [18]. Some exploit model sparsity by designing specialized hardware modules to accelerate computations involving unstructured or structured sparsity. Others focus on quantization; for example, FACT [18] demonstrates the benefits of mixed-precision quantization in improving Transformer inference speed.

Unlike earlier works that primarily targeted BERT or ViT models, recent research has shifted toward accelerating larger LLMs. Projects like OPAL [21], OliVe [20], and Oltron [22] adopt specialized quantization schemes to accommodate the increased complexity. FlightLLM [15] designs an accelerator supporting both mixed precision and sparsity in matrix multiplication, but this introduces additional hardware overhead and may not integrate well with systolic arrays.

III. XNOR-SHIFT ENCODING (XSE)

A. Basic XSE Format Principles

In this section, we present the XNOR-Shift Encoding (XSE) format, a simple yet effective method that provides customizable bit-width configurations for enhanced accuracy while preserving the DSP-free characteristics of Binarized Neural Networks (BNNs) for matrix computation. As depicted in Fig. 3(a) (bottom), shift operations are approximately 8× more resource and energy efficient than multiplication and addition units in low-bit computations, making the XSE format highly suitable for hardware-optimized low-bit quantization in LLMs.

The XSE format enables computational efficiency by replacing multiplications with XNOR and shift operations. As shown in Fig. 3(a) (top), the 3-bit XSE format encodes two data values using a straightforward base-2 scheme. This approach transforms multiplication into an XNOR operation applied to the sign bit and a shift operation applied to the shift bit, significantly reducing computational overhead. Furthermore, this basic framework can be

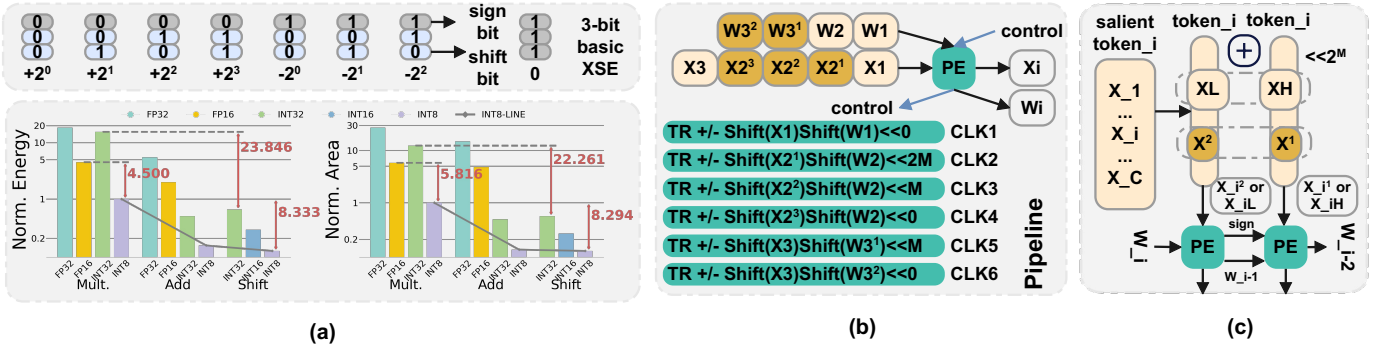


Fig. 3. Principles of XSE: (a) Basic 3-bit XSE format, along with energy and area comparisons between multiplication, addition, and shift operations; (b) Computation flow for channel-wise extended XSE; (c) Calculation principle for salient token with extended XSE.

extended to higher bit-width configurations, providing a broader range and improved precision.

B. Extended XSE for Outliers

Traditional methods that treat outliers as sparsity, often introduce significant overhead in hardware resources and runtime bandwidth. To address these limitations, we propose an extended XSE format that balances computation time and hardware efficiency. This format aligns seamlessly with the bandwidth requirements of the basic XSE format and integrates effectively with systolic array computations. With minimal hardware costs, the extended XSE format enables computations to be distributed across multiple clock cycles.

$$(-1)^{\text{sign}} \times (1 \ll 2N + I_H \ll N + I_L \ll 0) \ll \text{shift}.$$

The extended $3 \times N$ -bit XSE format adopts a floating-point-like representation. This scalable-width mantissa configuration provides a broader range. Extending an N -bit XSE format to an $M \times N$ -bit XSE format involves designating $(M - 1) \times N$ bits as an unsigned Int, $N - 1$ bits for the exponent shift, and one bit for the sign. This enhancement significantly improves the precision of outliers, effectively capturing critical data that influence model accuracy.

The computation flow for multiplying extended $M \times N$ -bit XSE data X_2 with basic N -bit XSE data W_2 is illustrated in Fig. 3(b), where the extended XSE format is applied to the channel-wise of activations: (1) The 1-bit sign and 2-bit shift of X_2^1 and W_2 are introduced. A flag shifts the data by six bits before it enters the adder, partially computing the product. (2) The upper 3-bit mantissa of X_2^2 is transmitted, applying an additional 3-bit shift to compute the second component while W_2 remains queued for the next cycle. (3) The lower 3-bit mantissa of X_2^3 is processed, completing the computation, while W_2 continues to propagate through the pipeline.

As the pipeline advances, new data (X_2 and W_2) are sequentially fed into the computation units, enabling efficient and continuous processing. The extended XSE format significantly broadens the applicability of the basic XSE scheme, enabling precise handling of outliers in large models. It achieves high accuracy without necessitating substantial hardware modifications. Furthermore, the XSE format remains DSP-free and straightforward to extend, making it an ideal choice for enhancing the accuracy of BNNs.

IV. HARDWARE ADAPTIVE OUTLIER AND SPARSITY

Using only magnitude as a metric is insufficient, as it neglects their cumulative impact on the model's output. Inspired by Optimal Brain Damage (OBD) [35], previous methods for outlier quantization [8]

and sparsification [12], [14], [17] have used a sensitivity matrix to measure errors, formulated as:

$$\begin{aligned} \mathbb{S}_W &= (W - Q(W))^2 / \text{diag}[(2X^T X + \lambda I)^{-1}] \\ &\Rightarrow (W - Q(W))^2 \parallel X_{:,j} \parallel_2^2. \end{aligned}$$

$(2X^T X + \lambda I)^{-1}$ is the Hessian H , and λ is a dampening factor to prevent instability in the inverse computation. When $\lambda = 0$ and only the diagonal elements of H are retained, the $O(d^3)$ complexity of H^{-1} simplifies to the L2 norm of X with $O(d^2)$ complexity.

Activations X , often exhibiting high magnitudes at the channel level, are adjusted using a channel-wise bias term for activations:

$$\begin{aligned} XW &= (\hat{X} \odot S_{(1,C_{in})} + B_{(1,C_{in})}) \cdot (\hat{W} \oslash S_{(1,C_{in})}) \\ &= \hat{X} \times \hat{W} + \hat{B}_{(1,C_{out})}, \\ \mathbb{S}_W &= (\hat{W} - Q(\hat{W}))^2 \parallel \hat{X}_{:,j} \parallel_2^2, \\ \mathbb{S}_X &= (\hat{X} - Q(\hat{X}))^2 \parallel \hat{W}_{i,:} \parallel_2^2, \end{aligned}$$

where the channel-wise bias B represents the average channel value of X with dimensions (L, C_{in}) , while $\hat{B} = B \cdot W$ ensures that bias adjustments without affect final outputs. Inspired by [5]–[7], [10], channel-wise scaling S redistributes quantization difficulty. To compensate for irregular weight W distributions, $W + ab^T$ is used, where $W \in \mathbb{R}^{C_{in} \times C_{out}}$, $a \in \mathbb{R}^{C_{in}}$, and $b \in \mathbb{R}^{C_{out}}$. $Q(\cdot)$ uses the XSE format in Sec. III with additional learnable parameters α and β to control clipping ranges. During sparsity sensitivity, quantized values $Q(\hat{X})$ and $Q(\hat{W})$ are treated as zero. The required additional parameters are minimal and learnable, trained using a small calibration dataset. For activation sensitivity $\mathbb{S}_X$, we identify outlier channels using a metric that considers quantization error rather than solely magnitude or variance (e.g., the $3\text{-}\sigma$ rule in [20], [22]). This approach accounts for the influence of weights on activations, crucial for the XSE format. Unlike $\mathbb{S}_W$, which can be used to update weights [8], [14], [36], $\mathbb{S}_X$ assesses activation sensitivity to determine outlier channels only, as activations cannot be updated.

Considering $XShift$'s hardware constraints, outlier detection and pruning occur statically at the channel level, avoiding outliers multiplication within the same C_{in} . Quantify and inference separately for activation and weight on the conflicting outlier dimension to determine which one has a greater impact on the output result, in order to decide which can preserve this outlier channel. By organizing computations along the channel dimension, stepwise propagation is maintained, ensuring compatibility with the systolic array architecture. For both activations and weights, specific channels will increased bit widths, while less significant channels are pruned.

Additionally, specific outlier token support ensures accuracy for tokens requiring greater attention. Special tokens like $\langle \text{BOS} \rangle$ and

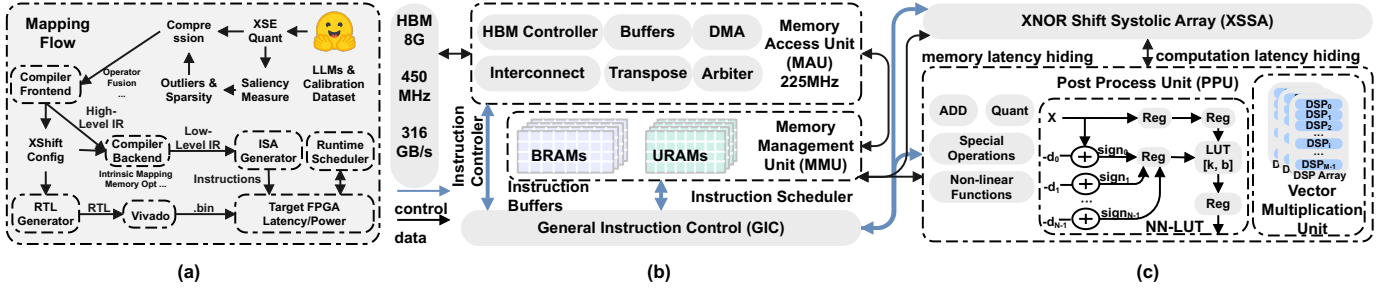


Fig. 4. Comprehensive architecture of *XShift*: (a) General mapping flow; (b) Overview of hardware architecture; (c) Details of the Post-Process Unit (PPU) featuring NN-LUT and a shared Vector Multiplication Unit.

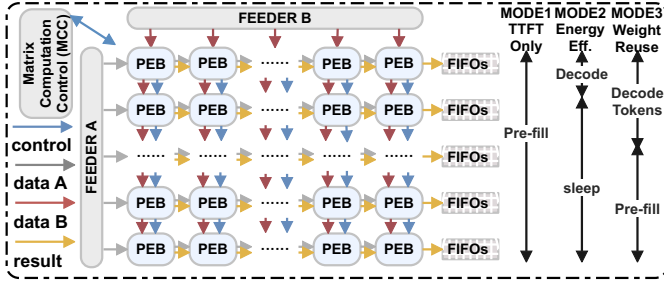


Fig. 5. Detailed architecture of XNOR Shift Systolic Array (XSSA) with support for all three computation modes.

$\langle \text{CLS} \rangle$, which contain global information critical for tasks in language generation and classification [37]. Fig. 3(c) illustrates the principle of expanding one token into M tokens with extended XSE activation values. This process maintains systolic array efficiency by passing XNOR results to subsequent PEs and summing M tokens to produce the outlier token result. For outlier weights, corresponding activation channels use basic XSE to avoid calculation conflicts. *XShift* supports N outlier tokens per K tokens depending on the systolic array size in FEEDER A.

V. HARDWARE ARCHITECTURE

A. *XShift* Mapping Flow

As illustrated in Fig. 4(a), the algorithmic optimizations detailed in Sec. III and Sec. IV enable the creation of a highly compressed model with superior accuracy. Once the model is optimized, it is processed through a compiler to generate the network graph pattern in High-Level Intermediate Representation (IR). This representation is then passed to the compiler backend for hardware-specific optimizations. The final Low-Level IR is subsequently translated by the ISA Generator into optimized instructions and model parameters, tailored for hardware deployment.

Recognizing the unique requirements of the Pre-fill and Decoding stages in LLMs, as well as the intermittent nature of user requests in edge applications, the *XShift* hardware design incorporates a parameter-sharing-driven scheduler and a versatile XNOR-Shift Systolic Array (XSSA), as depicted in Fig. 5. This architecture is capable of supporting three distinct computation modes, each designed to optimize efficiency and responsiveness.

When a user initiates a request, the system begins with the Pre-fill stage. During this phase, the scheduler activates the first XSSA computation mode, where all Processing Elements (PEs) within the XSSA are utilized to accelerate the process and minimize latency. Any additional user requests are queued during this time. Upon completion of the Pre-fill stage for the initial request, the system transitions to the Decoding stage, which demands less computational power and latency. In this second computation mode, only the

first layer of the XSSA is active, while other layers are idled to conserve static power. The processing frequency in this mode can be dynamically adjusted to meet latency requirements. For instance, maintaining a 200 ms interval per Decoding stage iteration aligns with typical English reading speeds of 300 words per minute.

If a new request arrives while a Decoding stage is in progress, the standard approach would require the new request to wait until the current Decoding stage completes. However, Decoding stage latency can vary significantly, often lasting several seconds depending on the token count, making this approach impractical. The *XShift* design mitigates this issue by allowing the Pre-fill stage of the new request to batch with the ongoing Decoding stage, thereby sharing weight parameters to reduce latency. Furthermore, multiple Decoding stages can be batched together for matrix multiplications involving the weight matrix. In this third computation mode, the scheduler ensures the ongoing Decoding maintains its token-per-minute output while maximizing parameter reuse during activation-weight multiplications. This strategy minimizes the wait time for new Pre-fill and ongoing Decoding stages, significantly improving overall system efficiency and responsiveness.

B. XNOR-Shift Systolic Array

To efficiently execute the matrix computations required by the *XShift* model, we introduce a 2-D XNOR-Shift Systolic Array (XSSA), meticulously designed to achieve both computational efficiency and scheduling requirements. The XSSA architecture is organized into K layers, each comprising M Processing Element Blocks (PEBs). Each PEB contains $n_1 \times n_2$ Processing Elements (PEs), resulting in a peak processing capacity of $K \times M \times n_1 \times n_2$ Multiply-Accumulate (MAC) operations per clock cycle. This systolic array structure optimizes data reuse and minimizes bandwidth demands, making it highly effective for parallel matrix multiplications.

Data is fed into the systolic array through two input feeders, FEEDER A and FEEDER B. FEEDER A provides input data at a bandwidth of $D_A \times K$ bits, while FEEDER B operates at $D_B \times M \times n_1 \times n_2$ bits, propagating data horizontally and vertically through the array. The Matrix Computation Control (MCC) unit oversees the pipeline operations, managing process initiation, pauses, and completion while also handling control signals. When a channel contains an outlier, the MCC dispatches specific flag to the PEs, with commands cascading through the array. Similarly, result timing follows a sequential pattern, with outputs being transmitted to the FIFO buffer at a bandwidth of $D_O \times K \times n_1 \times n_2$ bits. These outputs can then undergo dequantization or other forms of post-processing.

The design of the PEB strategically minimizes the systolic array depth M , which reduces waiting times and lowers intermediate register costs. Each PEB encompasses $n_1 \times n_2$ PEs that function simultaneously on the same clock cycle. As shown in Fig. 6, the PEs

Algorithm 1: Base-2 SoftMax Converter (BSMC) algorithm in *XShift*

Input : Result after QK^T as $\{a_0, \dots, a_{N-1}\}$ with a mask size of N , Scale value pre-compute $\omega_0 \times \log_2(e)$ and $\omega \times \log_2(e)$
Output: Quantified SoftMax output $\{s_0, \dots, s_{N-1}\}$, Scale value α

```

1  $\alpha \times \{s_0, \dots, s_{N-1}\} = \frac{\exp(\hat{\omega} \times a_i)}{\exp(\omega_0 \times a_0) + \sum_{i=1}^{N-1} \exp(\omega \times a_i)}$ ;
2 where  $\hat{\omega} \leftarrow \omega_0$ , if  $i = 0$ ;  $\hat{\omega} \leftarrow \omega$ , otherwise;
3 Initialize  $I_{max} = 0$ ,  $SUM = 0$ ;
4 for  $i \leftarrow 0$  to  $N - 1$  do
5    $x_i \leftarrow a_i \times (\hat{\omega} \times \log_2(e))$ ; /* Convert  $e^{a_i \times \hat{\omega}}$  to  $2^{x_i}$  */
6    $I_i, d_i \leftarrow x_i$ ; /*  $I_i$  is the integer part,  $d_i$  is the decimal part */
7    $I_{max} = \max(I_{max}, I_i)$ ,  $i \neq 0$ ;  $k_i, b_i \leftarrow d_i$ ; /* Approximate  $2^{d_i} = d_i \times k_i + b_i$  */
8 for  $i \leftarrow 0$  to  $N - 1$  do
9    $s_i \leftarrow (k_i \times d_i + b_i) \gg (I_{max} - I_i)$ ;  $SUM = SUM + s_i$ ;
10   $s_0 \leftarrow XSE_{extend}(s_0)$ , if  $i = 0$ ;  $s_i \leftarrow UINT(s_i, 2M)$ , if  $i \in window$ ;  $s_i \leftarrow UINT(s_i, M)$ , otherwise;
11  $\alpha = 1/SUM$ ;
12 return  $\{s_0, \dots, s_{N-1}\}, \alpha$ ;
```

are designed with simplicity and efficiency. During each cycle, the PEs receive input data A and B , apply an XNOR operation on their sign bits, and execute bit-shift operations on the remaining bits. The resulting values are then combined through addition or subtraction, determined by the XNOR outcome. If the outlier control signal flags A as an outlier, additional bits are introduced in subsequent cycles, with corresponding bit-shift operations and accumulations conducted as necessary. This approach minimizes hardware adjustments while ensuring efficient processing. The processed data, results, and control signals are then propagated to subsequent stages for reuse, maintaining a seamless flow of computations.

C. Base-2 SoftMax Converter

The SoftMax function, a critical component in LLM post-processing, presents significant challenges due to its high computational complexity and stringent precision requirements, particularly when computing exponentials such as $e^{s \cdot A}$. Without simplifications, these calculations demand substantial resources and result in a wide dynamic range of outputs, exacerbating errors from piecewise linear approximations more than other functions. Additionally, the strictly positive nature of SoftMax outputs and the variable input lengths in LLMs necessitate dynamic quantization to prevent computational losses in subsequent stages.

To address these challenges, we propose a hardware-efficient solution: the Base-2 SoftMax Converter (BSMC). This approach achieves both accuracy and efficiency in SoftMax calculations without requiring model retraining. The BSMC method, outlined in Algorithm 1, begins by scaling the XSSA output, a_i , by a factor ω , to compute $e^{s \cdot A}$. Instead of directly calculating $e^{\omega \cdot a_i}$, ω is replaced by $\omega \cdot \log_2(e)$, transforming $e^{\omega \cdot a_i}$ into $2^{\omega \cdot \log_2(e) \cdot a_i}$. This precomputation of $\omega \cdot \log_2(e)$ significantly improves computational efficiency.

It is worth noting that the first token, represented as $\langle BOS \rangle$, is assigned its own scaling factor, ω_0 , and is quantified using an extended XSE format. This token is critical for downstream computations and has a significant impact on overall results. For each element a_i , we calculate $\omega \cdot \log_2(e) \cdot a_i$, decomposing it into an integer part I_i and a decimal part d_i . During this process, I_{max} , the maximum integer value across all I_i (excluding the first token), is identified. The expression $2^{\omega \cdot \log_2(e) \cdot a_i}$ is then simplified to $2^{d_i} \ll I_i$, where $d_i \in [0, 1)$ can be efficiently approximated using NN-LUT [38]. The NN-LUT employs parameters k_i and b_i for piecewise linear

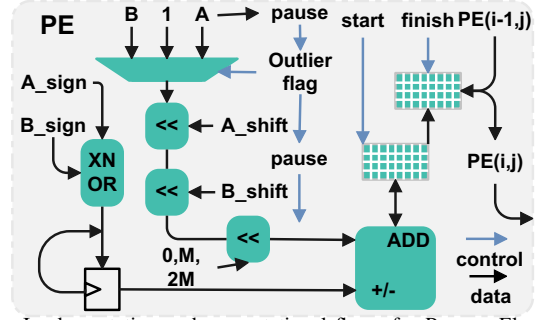


Fig. 6. Implementation and computational flow of a Process Element (PE) within the XSSA.

interpolation, approximating 2^{d_i} as $k_i \cdot d_i + b_i$. This restricted range for d_i ensures high precision.

In subsequent quantization, the exponent is adjusted by subtracting I_{max} , preserving the SoftMax computation while simplifying each term s_i as $(k_i \cdot d_i + b_i) \gg (I_{max} - I_i)$. The cumulative sum, SUM, is computed concurrently. Instead of directly normalizing s_i by multiplying with $\frac{1}{SUM}$, we leverage the earlier adjustment to quantize s_i as an unsigned integer. This eliminates the need for a sign bit, extends the range of representable values, and allows for higher precision within a predefined window. The scaling factor $\frac{1}{SUM}$ is then designated as row-specific and seamlessly integrates with subsequent matrix computations involving V . Importantly, the transposed V matrix remains free of outlier quantization, ensuring full compatibility with the XSSA matrix multiplication framework.

VI. EXPERIMENTAL RESULTS

Models and Datasets. The *XShift* framework employs advanced compression techniques from Sections III and IV to optimize LLaMA family models [2], including LLaMA, LLaMA-2, and the latest LLaMA-3. We evaluate perplexity (PPL) as the primary metric for language generation tasks, using the WikiText2 [39] and C4 [40] datasets. Additionally, zero-shot task performance is evaluated via the lmevaluation-harness framework [41], which covers a range of tasks including PiQA, ARC, BoolQ, and HellaSwag, to assess the generalization capabilities of *XShift*.

Experimental Settings. For software-based evaluations, CUDA Toolkit 11.8 and PyTorch 2.0.0 were used. Calibration involved 128 samples randomly extracted from WikiText2, each consisting of 2048 tokens. Models were fine-tuned over 10–40 epochs, targeting optimal sparsity and quantization levels through a structured pipeline. For hardware-based evaluations, *XShift* is deployed on the Xilinx Alveo U50 and U280 FPGAs. The U50 FPGA, a cost-effective alternative to the U280 (\$5000), achieves reduced resource consumption while maintaining high efficiency. Hardware implementations were written in SystemVerilog and operated at 225 MHz for optimal memory bandwidth. The synthesis is conducted using Xilinx Vitis v2023.1 and Vivado toolchains. Comparisons were drawn against Nvidia V100 and A100 GPUs using PyTorch-based implementations (GPU-native) and SmoothQuant-optimized models (GPU-opt).

A. Accuracy Analysis

Quantized Model Performance: The performance of *XShift* is first assessed on WikiText2 and C4 datasets. Table I presents the inference operations (OPs) and perplexity (PPL) results. Compared to state-of-the-art methods such as BiLLM [3] (binarization), Wanda [12] (sparsity), I-LLM [9] (low-precision quantization), Tender [19] and Oltron [25] (software-hardware co-design), *XShift* demonstrates the lowest PPL across both datasets.

TABLE I

NUMBER OF INFERENCE OPERATIONS (OPS) AND PERPLEXITY RESULTS (PPL) OF QUANTIZED MODEL ON (WIKITEXT2 / C4) DATASETS (LOWER IS BETTER).

Method	FP16		BiLLM [3]		Wanda [12]		I-LLM [9]		Tender [19]		Oltron [22]		XShift ₄		XShift ₃	
Models	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓	OPs↓	PPL↓
LLaMA-7B	14.64T	5.68/7.08	2.03T	35.04/-	7.87T	8.57/-	0.92T	9.10/12.33	0.92T	23.85/38.09	0.94T	11.67/15.21	0.84T	8.11/11.65	0.59T	8.42/12.44
LLaMA-13B	28.05T	5.09/6.61	3.52T	15.14/-	14.89T	7.40/-	1.76T	7.99/10.96	1.76T	13.68/28.24	1.80T	8.20/10.84	1.58T	7.45/9.36	1.11T	7.78/9.44
LLaMA2-7B	14.64T	5.47/6.97	2.02T	23.48/-	7.87T	7.97/-	0.92T	10.44/12.92	0.92T	36.47/114.44	-	-/-	0.76T	8.01/10.64	0.45T	8.32/11.41
LLaMA2-13B	28.05T	4.88/6.46	3.51T	16.77/-	14.89T	6.55/-	1.76T	9.76/12.57	1.76T	55.08/208.76	-	-/-	1.44T	8.22/10.31	0.85T	9.65/11.88
LLaMA3-8B	16.48T	6.14/8.88	2.16T	28.3/2.9E2	-	-/-	1.04T	21.19/30.9	-	-/-	-	-/-	0.93T	14.42/18.45	0.66T	15.54/20.75

TABLE II

ACCURACY RESULT ON VARIOUS ZERO-SHOT TASKS UNDER 4-BIT WEIGHT AND ACTIVATION.

Model	Method	PtQA	ARC-e	ARC-c	BoolQ	HellaSwag	Winogrande	Avg.
LLaMA-13B	FP16	79.10	59.89	44.45	68.01	76.21	70.31	66.32
	OmniQuant [6]	69.69	47.30	33.10	62.84	58.96	55.80	54.62
	I-LLM [9]	67.95	48.15	34.47	62.29	63.13	59.98	55.99
	XShift	73.88	52.40	40.96	66.44	69.40	65.23	61.39
LLaMA2-13B	FP16	79.05	57.91	44.19	69.02	76.60	69.69	66.07
	OmniQuant [6]	67.08	45.66	32.25	63.73	58.39	54.61	53.62
	I-LLM [9]	68.00	45.74	30.97	64.55	60.62	54.22	54.01
	XShift	73.47	51.21	39.45	66.76	68.23	62.77	60.32

TABLE III

COMPREHENSIVE RESOURCE UTILIZATION COMPARISON OF VARIOUS FPGA-BASED DESIGNS.

Method	Platform	LUT	FF	BRAM	URAM	DSP
DFX [16]	Alveo U280	520K	1107K	1192	104	3533
FlightLLM [15]	Alveo U280	574K	943K	1252	792	6345
XShift	Alveo U280	654K	820K	935	662	356
	Alveo U50	430K	512K	564	448	224

For instance, on smaller models, *XShift* outperforms Oltron significantly, achieving up to 3 lower PPL. Against BiLLM, *XShift* achieves nearly 50% lower PPL while maintaining fewer OPs, highlighting the effectiveness of its low-precision quantization for both weights and activations. The 4-bit *XShift*₄ variant operates at under 1T OPs for 2048-token inputs, making it ideal for edge deployments, while the compact 3-bit *XShift*₃ variant reduces computational complexity to approximately 0.5T OPs, maintaining competitive PPL performance.

Challenging LLaMA-3 Model: The LLaMA-3 model, known for its quantization difficulties, also highlights *XShift*'s robustness. Compared to I-LLM, *XShift* achieves up to 5 fewer PPL, maintaining similar OP counts. This underscores its ability to balance efficiency and accuracy even under challenging conditions.

Zero-Shot Task Evaluation: Table II highlights zero-shot task performance on LLaMA2-13B under 4-bit weight and activation settings. *XShift* consistently achieves the highest accuracy across tasks, demonstrating a 6.3% improvement in average accuracy on LLaMA-13B and LLaMA2-13B models with superior generalization capabilities. These improvements are attributed to *XShift*'s ability to identify and preserve critical outliers, ensuring robust performance across diverse tasks.

B. Hardware Performance and Utilization

Resource Utilization: Table III compares resource usage across FPGA-based designs, including DFX [16] and FlightLLM [15]. *XShift* achieves the lowest resource utilization, requiring only 224 DSPs on the U50 FPGA and 356 DSPs on the U280 FPGA and up to 10× fewer DSPs compared to alternative designs. This efficiency is enabled by the XNOR-Shift Systolic Array (XSSA), which eliminates traditional DSP constraints.

Latency and Energy Efficiency: Fig. 7 illustrates the normalized latency and energy of *XShift* compared to DFX, FlightLLM, and GPU-based implementations. *XShift* achieves a 4.17× improvement

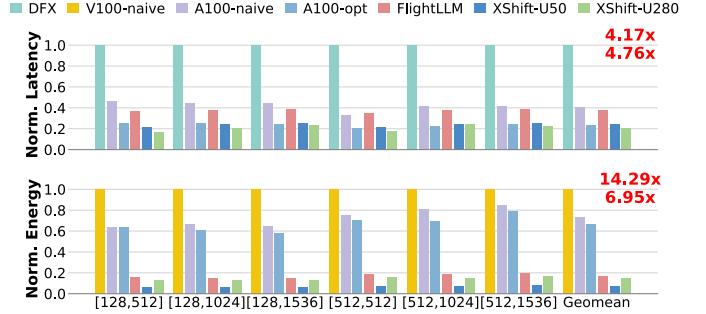


Fig. 7. Normlized latency and energy of *XShift* compared with other approaches. The horizontal axis represents [Pre-fill size, Decode size].

in latency and up to 14.29× better energy efficiency compared to the V100 GPU. Compared to the U280 FPGA implementation, the U50 FPGA achieves nearly 2× higher energy efficiency while maintaining comparable latency, emphasizing its suitability for edge deployments.

Edge Suitability: The reduced resource consumption and power requirements of *XShift* enable its deployment on cost-effective platforms like the Alveo U50 FPGA with a much better energy efficiency, making it highly practical for edge environments. Its optimized hardware-software co-design ensures that it not only meets but exceeds the performance of more resource-intensive solutions, such as FlightLLM, across both speed and energy metrics.

XShift establishes itself as a leading solution for quantizing and sparsifying LLMs, achieving state-of-the-art accuracy and efficiency. Its ability to combine low PPL, minimal OPs, and superior hardware utilization makes it a robust choice for deploying next-generation LLMs in resource-constrained environments. These results validate *XShift* as a comprehensive framework capable of delivering optimal performance on both software and hardware fronts.

VII. CONCLUSION

We introduce *XShift*, an algorithm-hardware co-design framework for efficient binarized LLM inference on FPGAs. By leveraging the XNOR-Shift Encoding (XSE) format and Hardware Adaptive Outlier and Sparsity (HAOS) techniques, *XShift* achieves significant compression while maintaining high model accuracy. Our framework integrates a custom-designed XNOR-Shift Systolic Array (XSSA) and a Base-2 SoftMax Converter (BSMC) to address the unique computational and memory demands of LLM inference, particularly in edge environments. Experimental results demonstrate *XShift*'s superiority in accuracy, computational efficiency, and energy savings, outperforming state-of-the-art methods on benchmarks like Wiki-Text2 and C4. Its deployment on the Xilinx Alveo U280 and U50 FPGA highlights its hardware efficiency, making it ideal for edge environments.

By addressing the challenges of LLM binarization and tailoring hardware designs to the specific needs of Pre-fill and Decoding stages, *XShift* establishes itself as a robust solution for deploying LLMs in resource-constrained environments.

REFERENCES

- [1] A. Radford, "Improving language understanding by generative pre-training," 2018.
- [2] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [3] W. Huang, Y. Liu, H. Qin, Y. Li, S. Zhang, X. Liu, M. Magno, and X. Qi, "BiLLM: Pushing the limit of post-training quantization for LLMs," *arXiv preprint arXiv:2402.04291*, 2024.
- [4] H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei, "BitNet: Scaling 1-bit transformers for large language models," *arXiv preprint arXiv:2310.11453*, 2023.
- [5] C. Zeng, S. Liu, Y. Xie, H. Liu, X. Wang, M. Wei, S. Yang, F. Chen, and X. Mei, "ABQ-LLM: Arbitrary-bit quantized inference acceleration for large language models," *arXiv preprint arXiv:2408.08554*, 2024.
- [6] W. Shao, M. Chen, Z. Zhang, P. Xu, L. Zhao, Z. Li, K. Zhang, P. Gao, Y. Qiao, and P. Luo, "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [7] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 38 087–38 099.
- [8] T. Dettmers, R. A. Svirskiy, V. Egiiazarian, D. Kuznetsov, E. Frantar, S. Ashkboos, and *et al.*, "SpQR: A sparse-quantized representation for near-lossless LLM weight compression," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [9] X. Hu, Y. Cheng, D. Yang, Z. Yuan, J. Yu, C. Xu, and S. Zhou, "I-LLM: Efficient integer-only inference for fully-quantized low-bit large language models," *arXiv preprint arXiv:2405.17849*, 2024.
- [10] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration," *Proceedings of Machine Learning and Systems (MLSys)*, vol. 6, pp. 87–100, 2024.
- [11] Z. Guan, H. Huang, Y. Su, H. Huang, N. Wong, and H. Yu, "APTQ: Attention-aware post-training mixed-precision quantization for large language models," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [12] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [13] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 21 702–21 720, 2023.
- [14] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," in *International Conference on Machine Learning (ICML)*. PMLR, 2023, pp. 10 323–10 337.
- [15] S. Zeng, J. Liu, G. Dai, X. Yang, T. Fu, H. Wang, W. Ma, H. Sun, S. Li, Z. Huang *et al.*, "FlightLLM: Efficient large language model inference with a complete mapping flow on fpgas," in *Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays (FPGA)*, 2024, pp. 223–234.
- [16] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "DFX: A low-latency Multi-FPGA appliance for accelerating transformer-based text generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022.
- [17] X. Lin, H. Tian, W. Xue, L. Ma, J. Cao, M. Zhang, J. Yu, and K. Wang, "FLAME: Fully leveraging moe sparsity for transformer on FPGA," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [18] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, "Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023.
- [19] J. Lee, W. Lee, and J. Sim, "Tender: Accelerating large language models via tensor decomposition and runtime requantization," *arXiv preprint arXiv:2406.12930*, 2024.
- [20] C. Guo, J. Tang, W. Hu, J. Leng, C. Zhang, and *et al.*, "Olive: Accelerating large language models via hardware-friendly outlier-victim pair quantization," in *Proceedings of the 50th Annual International Symposium on Computer Architecture (ISCA)*, 2023, pp. 1–15.
- [21] J. Koo, D. Park, S. Jung, and J. Kung, "OPAL: Outlier-preserved microscaling quantization accelerator for generative large language models," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [22] C. Xue, C. Zhang, X. Jiang, Z. Gao, Y. Lin, and G. Sun, "Oltron: Algorithm-hardware co-design for outlier-aware quantization of LLMs with inter-/intra-layer adaptation," in *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*, 2024, pp. 1–6.
- [23] Z. Liu, B. Oguz, A. Pappu, L. Xiao, S. Yih, M. Li, R. Krishnamoorthi, and Y. Mehdad, "BiT: Robustly binarized multi-distilled transformer," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 14 303–14 316, 2022.
- [24] P.-H. C. Le and X. Li, "BinaryViT: Pushing binary vision transformers towards convolutional models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [25] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for Transformer-Based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [26] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative LLM inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [27] M. Brysbaert, "How many words do we read per minute? a review and meta-analysis of reading rate," *Journal of memory and language*, vol. 109, p. 104047, 2019.
- [28] E. Park, D. Kim, and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 688–698.
- [29] A. H. Zadeh, I. Edo, O. M. Awad, and A. Moshovos, "GOBO: Quantizing attention-based NLP models for low latency and energy efficient inference," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 811–824.
- [30] S.-y. Liu, Z. Liu, X. Huang, P. Dong, and K.-T. Cheng, "LLM-FP4: 4-bit floating-point quantized transformers," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 592–605.
- [31] Y. An, X. Zhao, T. Yu, M. Tang, and J. Wang, "Fluctuation-based adaptive structured pruning for large language models," in *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, vol. 38, no. 10, 2024, pp. 10 865–10 873.
- [32] J. Guo, J. Wu, Z. Wang, J. Liu, G. Yang, Y. Ding, R. Gong, H. Qin, and X. Liu, "Compressing large language models by joint sparsification and quantization," in *Forty-first International Conference on Machine Learning (ICML)*, 2024.
- [33] Y. Zhang, J. Pan, X. Liu, H. Chen, D. Chen, and Z. Zhang, "FracBNN: Accurate and FPGA-efficient binary neural networks with fractional activations," in *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays (FPGA)*, 2021, pp. 171–182.
- [34] S. Zhou, S. Meng, H. Tian, J. Yu, and K. Wang, "Edge-BiT: Software-hardware co-design for optimizing binarized transformer networks inference on edge FPGA," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2025.
- [35] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," *Advances in neural information processing systems (NeurIPS)*, vol. 2, 1989.
- [36] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, "GPTQ: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [37] M. Zaheer, G. Guruganesh, K. A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang *et al.*, "Big bird: Transformers for longer sequences," *Advances in neural information processing systems (NeurIPS)*, vol. 33, pp. 17 283–17 297, 2020.
- [38] J. Yu, J. Park, S. Park, M. Kim, S. Lee, D. H. Lee, and J. Choi, "NN-LUT: Neural approximation of non-linear operations for efficient transformer inference," in *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, 2022, pp. 577–582.
- [39] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016.
- [40] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *arXiv e-prints*, 2019.
- [41] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, and *et al.*, "A framework for few-shot language model evaluation," 07 2024. [Online]. Available: <https://zenodo.org/records/12608602>